



Context Course documentation

Quiz 1: Hook Fundamentals ▾



Quiz 1: Hook Fundamentals

[Copy page](#)

Test your understanding of hooks, the events they cover, and how they differ across Claude Code, Codex, OpenCode, and Pi.

Question 1: What is a hook?

- ☐ A hook is a model prompt that reminds the agent to run a command
- ☒ A hook is a user-defined handler that runs at a deterministic point in the agent's lifecycle, independent of the model's cooperation

Correct! Correct! Hooks fire on events like PreToolUse and Stop, and they run whether or not the model 'wanted' them to.

- ☐ A hook is a way to package skills, MCP servers, and agents into a distributable bundle
- ☐ A hook is a kind of subagent that runs in parallel with the main agent

[Submit](#)

You got all the answers!

Question 2: How do hook surfaces differ across platforms?

- ☒ Claude Code uses PascalCase JSON events (PreToolUse, Stop); Codex uses a smaller PascalCase JSON set behind a feature flag; OpenCode uses TS/JS plugin modules; Pi uses TS/JS extensions with lower_snake_case events like tool_call

Correct! Correct! The four platforms share the same lifecycle but differ in syntax and surface.

- ☐ All four platforms use the exact same settings.js
- ☐ OpenCode exposes hooks through JSON config

[Ask HuggingChat](#)

- ☐ Only Claude Code supports HTTP hooks; Codex, OpenCode, and Pi cannot send events to an HTTP receiver

You got all the answers!

Question 3: Which lifecycle events matter most?

- ☐ PreToolUse fires after the tool returns, so you can log the result
- ☒ UserPromptSubmit fires when the user submits a prompt, before the model sees it — ideal for injecting extra context

Correct! Correct! This is the right event for prepending or modifying the prompt. Claude Code and Codex support UserPromptSubmit directly; Pi's nearest equivalent is `before_agent_start`.

- ☐ Stop fires only at session end, not per turn
- ☐ PostToolUse in Codex fires for every possible tool path, just like in Claude Code

You got all the answers!

Question 4: How do hooks block or modify actions?

- ☒ A Claude Code command hook exits with code 2 and writes a reason to stderr to block the action

Correct! Correct! Exit 2 + stderr is the documented blocking signal; on PreToolUse it blocks the tool call, on UserPromptSubmit it erases the prompt.

- ☐ A Claude Code hook returns HTTP 500 to block the action
- ☐ An OpenCode hook writes to stdin to block a tool call
- ☐ Hooks cannot block or modify tool calls; they are observe-only

You got all the answers!

Question 5: When should you use hooks in

 Ask HuggingChat

- ☐ Hooks replace skills: once you have hooks, you don't need to write skills anymore
- ☒ Hooks are the right surface when you need deterministic behavior — like 'always run the linter after an Edit' — that shouldn't depend on the model remembering

Correct! Correct! That's exactly what hooks are for: turning conventions into code so the runtime enforces them.

- ☐ Hooks are mainly for user interface theming
- ☐ Hooks only make sense for single-agent workflows

Submit

You got all the answers!

Summary

If you got 4-5 correct, you understand the core hook model well enough to start wiring real guardrails and observability. If not, review the event lifecycle and the platform comparison before continuing.

Key Takeaways

- Hooks are deterministic runtime handlers, not prompts or plugins
- Claude Code, Codex, OpenCode, and Pi expose similar lifecycle moments with different configuration surfaces
- Hooks are the right tool when behavior must happen every time rather than relying on the model to remember

Next Steps

Next, wire hooks into a real dashboard so you can see agent activity live.

[Update on GitHub](#)

 Ask HuggingChat



← Hook Events and the Agent Lifecycle

Hands-On: Agent Activity Dashboard with Gradio →



Ask HuggingChat

